



2. 과목 · 커리큘럼 · 수강신청

담당: 조아진

개요

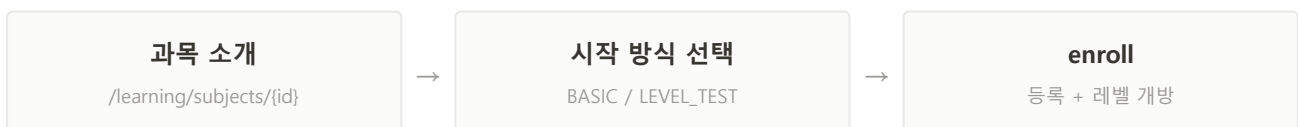
학습을 시작하려면 먼저 과목을 고르고 등록해야 한다. 등록 방식(기초 시작 / 레벨 테스트)에 따라 어느 레벨이 열린 채로 시작하는지가 갈린다.

- 구현 파일: `learning/controller/SubjectController.java`, `learning/service/EnrollmentService.java`, `learning/service/CurriculumService.java`

핵심 포인트

- 수강 등록은 `(user, subject)` UNIQUE 기준으로 **먹등**하게 — 같은 과목을 다시 눌러도 행이 늘지 않고 기존 등록을 그대로 돌려준다
- 시작 방식이 `BASIC` 이면 최저 레벨 `BRONZE` 를 `ENROLLMENT` 출처로 즉시 열고, `LEVEL_TEST` 면 채점 결과가 판정 등급까지 열어 준다
- "열린 레벨이 하나도 없음"을 곧바로 *레벨 테스트를 봐야 함*으로 해석한 게 **무한 리다이렉트**의 원인이었다 — `start_mode` 와 프로필 레벨을 함께 보도록 고쳤다
- 학습 가능 여부의 단일 기준은 `user_level_unlocks` — 프로필이 GOLD인데 unlock이 없으면 **화면은 GOLD인데 학습은 403**이 되는 모순이 생긴다
- 행성 안 레슨 목록은 `required_for_completion` 이 커진 활성 레슨만 완료 판정에 센다

1. 수강 신청과 시작 방식



`(user, subject)` UNIQUE 기준으로 먹등하게 만들었다. 같은 과목을 다시 눌러도 행이 늘지 않고 기존 등록을 그대로 돌려준다.

위치: `learning/service/EnrollmentService.java` 54~77행 (enroll)

```
/**
 * 과목을 수강 신청한다. (user, subject) UNIQUE 기준으로 먹등 - 이미 신청한 과목이면 기존 행을 반환한다.
 * startMode가 BASIC이면 최저 레벨(BRONZE)을 함께 연다. LEVEL_TEST면 레벨을 열지 않는다
 * (레벨 테스트 채점 결과가 판정 등급까지 열어 준다).
 */
```

```

@Transactional
public UserSubjectEnrollment enroll(Long userId, Long subjectId, String startMode) {
    UserSubjectEnrollment existing =
        enrollmentMapper.findByUserAndSubject(userId, subjectId).orElse(null);
    if (existing == null) {
        existing = new UserSubjectEnrollment();
        existing.setUserId(userId);
        existing.setSubjectId(subjectId);
        existing.setStatus(STATUS_ACTIVE);
        existing.setStartMode(startMode);
        existing.setEnrolledAt(LocalDateTime.now());
        enrollmentMapper.insert(existing);
    }

    if (START_MODE_BASIC.equals(startMode)) {
        unlockService.unlock(userId, subjectId, LOWEST_LEVEL_CODE, SOURCE_ENROLLMENT, null);
    }
    return existing;
}

```

- **BASIC** (기초부터) → 최저 레벨 **BRONZE** 를 **ENROLLMENT** 출처로 함께 연다
- **LEVEL_TEST** → 여기서 레벨을 열지 않는다. 채점 결과가 판정 등급까지 열어 주기 때문

2. 레벨 테스트 무한 리다이렉트 수정

"열린 레벨이 하나도 없음"을 곧바로 *레벨 테스트를 봐야 함*으로 해석하던 게 문제였다. 기초 시작을 고른 계정이나 프로필 레벨만 심어 둔 계정이 **레벨 테스트로 계속 되돌려졌다**.

위치: learning/service/EnrollmentService.java 95~116행 (requiresLevelTest)

```

@Transactional
public boolean requiresLevelTest(Long userId, Long subjectId) {
    if (!unlockMapper.findByUserAndSubject(userId, subjectId).isEmpty()) {
        return false;
    }

    UserSubjectEnrollment enrollment =
        enrollmentMapper.findByUserAndSubject(userId, subjectId).orElse(null);
    if (enrollment == null) {
        return true;
    }

    boolean startedWithLevelTest = START_MODE_LEVEL_TEST.equals(enrollment.getStartMode());
    if (startedWithLevelTest && levelTestService.hasQuestions(subjectId)) {
        return true;
    }

    for (String levelCode : levelsUpTo(profileLevelOf(userId, subjectId))) {
        unlockService.unlock(userId, subjectId, levelCode, SOURCE_ENROLLMENT, null);
    }
    return false;
}

```

구분자	<code>start_mode</code> 뿐 — 프로필 <code>current_level_code</code> 는 NOT NULL DEFAULT라 미응시를 가려내지 못한다
복구	기초 시작인데 unlock만 없는 계정은 프로필 레벨까지 <code>levelsUpTo</code> 로 채워 연다(레벨 테스트 채점과 같은 규약)
미등록	수강 기록 자체가 없으면 복구 대상이 아니다 — 시작 전 사용자에게 레벨을 열어 주면 "잠긴 로드맵"이 성립하지 않는다

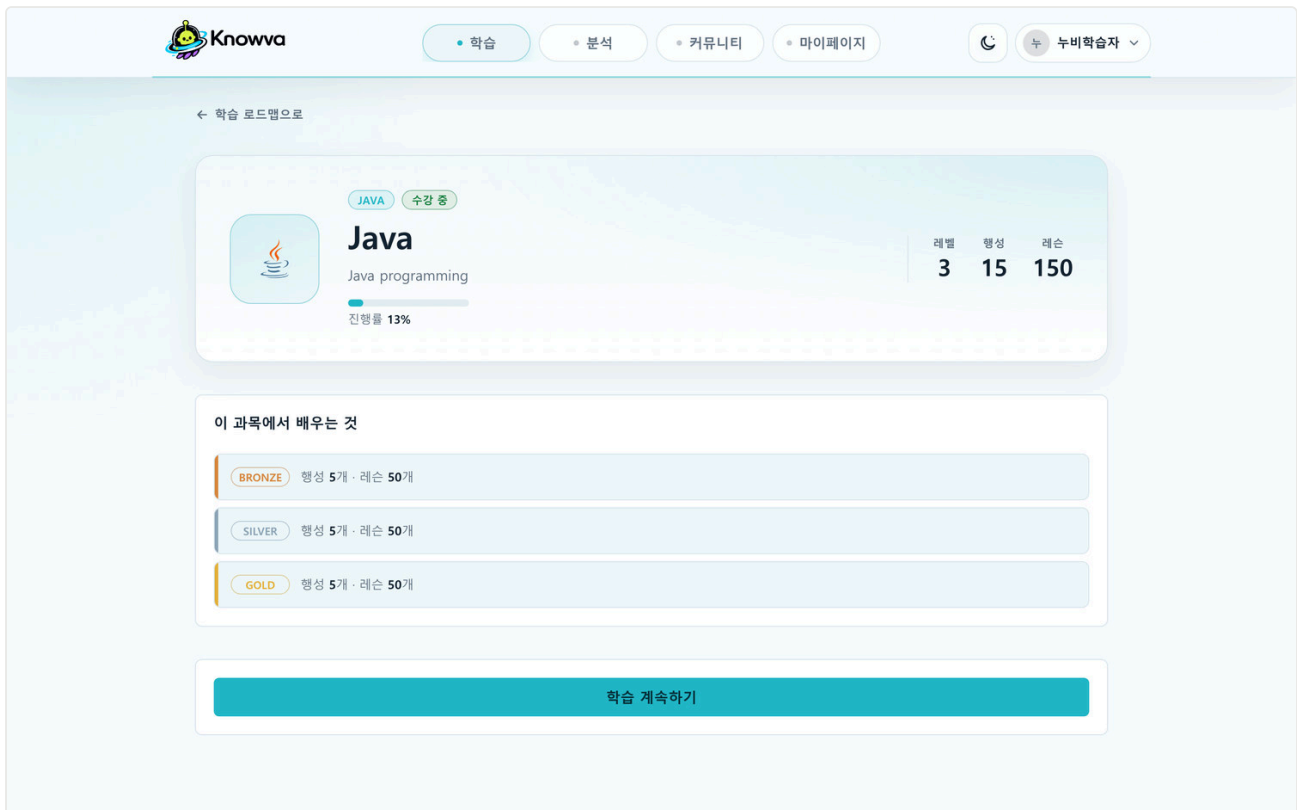
학습 가능 여부는 `user_level_unlocks` 만 본다. 프로필이 GOLD인데 unlock이 없으면 화면은 GOLD인데 학습은 403이 되는 모순이 생기므로, 두 곳을 같은 규약으로 맞추는 게 핵심이었다.

3. 행성 안의 레슨 구성

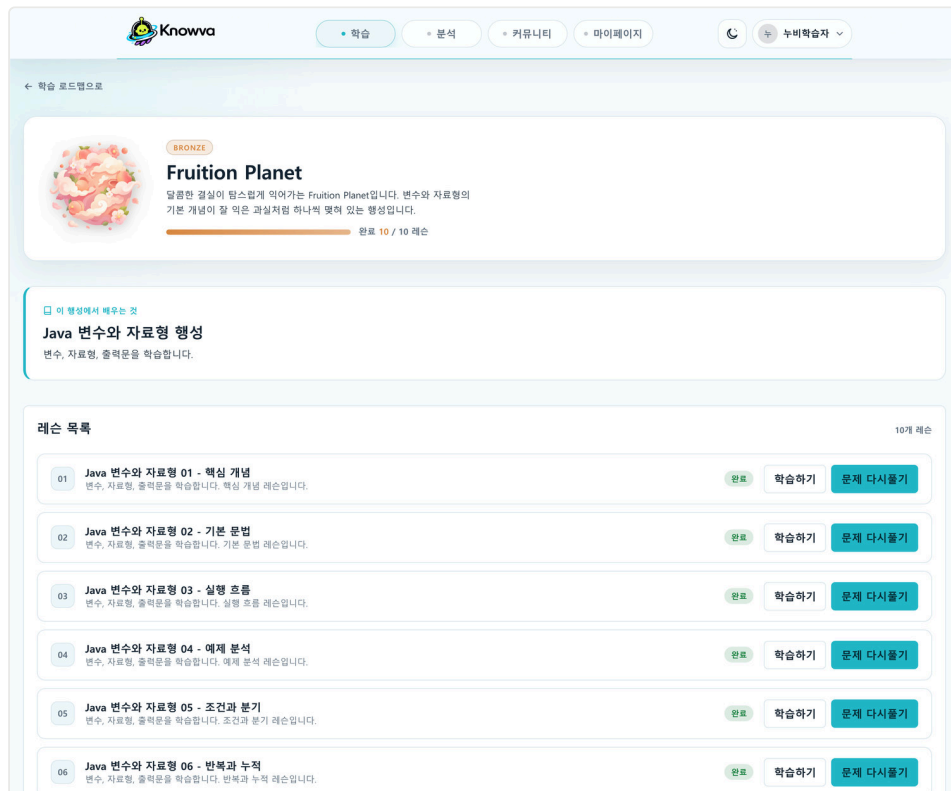
행성을 클릭하면 그 단원의 레슨 목록으로 들어간다. 상단은 행성 아트·이름·테마 스토리로 맥락을 주고, 아래에 실제 단위 설명과 레슨을 놓았다.

항목	기준
필수 레슨	<code>required_for_completion</code> 이 켜진 활성 레슨만 완료 판정에 센다
진입 차단	미수강 과목은 <code>AUTH_FORBIDDEN</code> (403) — <code>LessonService.requireEnrolled</code>
잠긴 레벨	열람은 허용하되 학습 액션은 막는다(팀 방침: 잠긴 로드맵도 볼 수 있어야 함)

시연 화면



과목 소개 화면. 수강 중이면 진행률과 **학습 계속하기**가, 미등록이면 시작 방식 선택이 나온다. 레벨별 행성/레슨 수는 실제 `curriculum_nodes` · `lessons` 집계다.



`/learning/nodes/{nodeId}/lessons` — 레벨 accent(BRONZE 주황)가 히어로와 배지에 함께 적용된다.